Computer Architecture and Operating Systems
COSC 22073/BECS 22233

Lesson 09: Introduction to Threads

Department of Statistics & Computer Science
Faculty of Science
University of Kelaniya

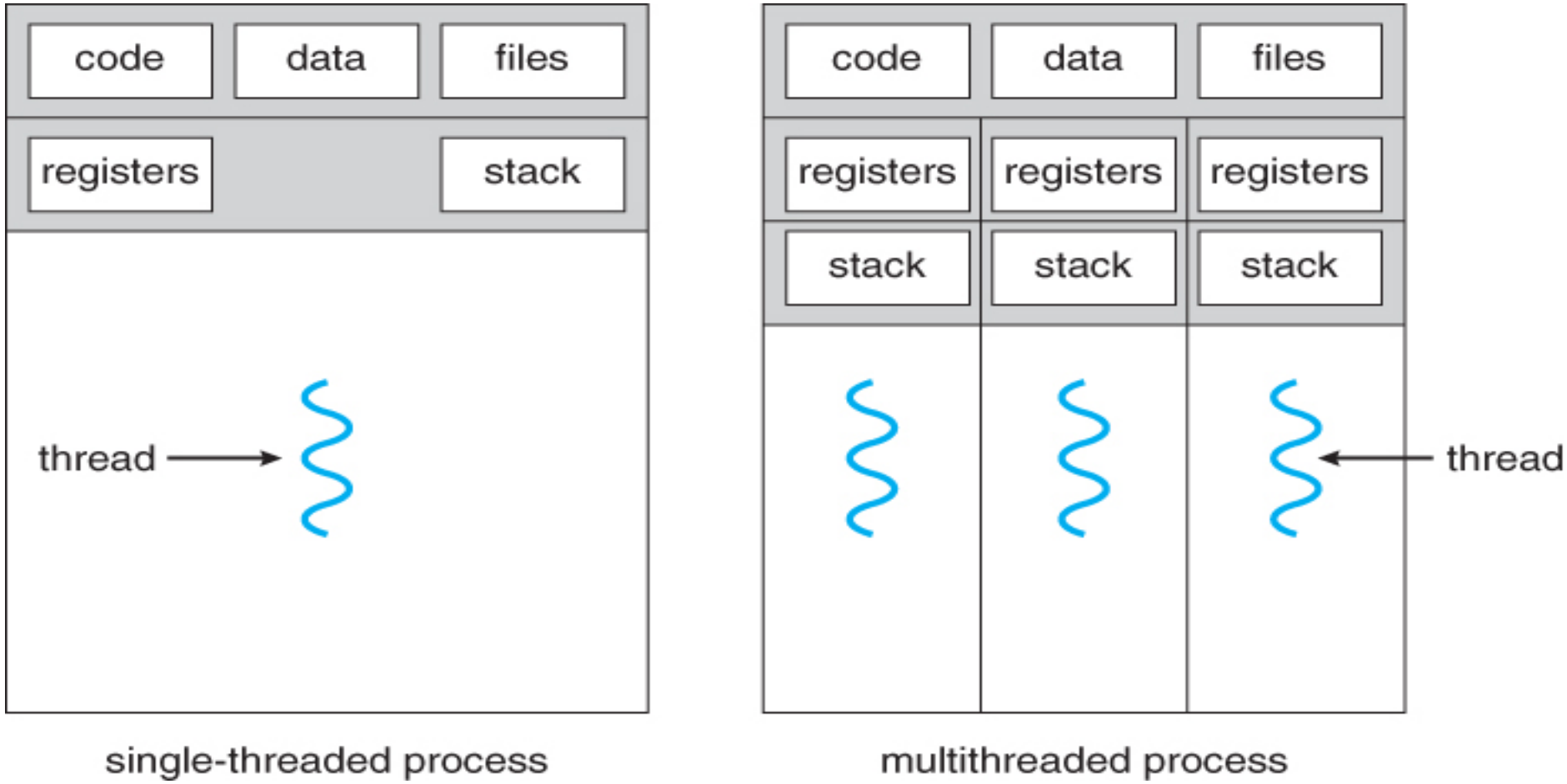
What is a Thread?

- A thread is a path of execution within a process. A process can contain multiple threads.
- Improving the responsiveness of user interfaces.
- Managing concurrent tasks. Threads within the same process can communicate and share data more easily than separate processes, which is a significant advantage in many applications.

Why do we need Thread?

- Most modern applications are multithreaded.
- Threads run within the application.
- Multiple tasks with the application can be implemented by separate threads:
  - Update display
  - Fetch data
  - Spell checking
  - Answer a network request
- Process creation is heavy-weight while thread creation is light-weight.
- Can simplify code, and increase efficiency.
- Kernels are generally multithreaded

Single and Multithreaded Processes



Process vs Thread

Classification	Process	Threads
Definition	Execution of any program is a process	Segment or subset of a process is a thread
Communication	It takes more time to communicate between the different processes.	It takes less time for the communications
Resources	It consumes more resources	It consumes fewer resources
Memory	The process does not share the memory and is carried out alone	Threads are used to share their memory
Data Sharing	Process does not share their data	Threads share their data
Size	The process consumes more size	Threads are lightweight
Execution error	One process will not affect any other process due to an error	Threads will not run if one thread has an error
Termination time	Process usually takes more time to terminate	Threads take less time to terminate

Benefits of Threads

- **Concurrency:** Threads allow multiple tasks to execute concurrently within the same program, which can lead to better utilization of available CPU resources and faster task execution. This is particularly important in multi-core and multi-processor systems.

- **Responsiveness:** Threads can make applications more responsive. For example, in a graphical user interface, one thread can handle user input and interface updates, while another thread can perform background tasks like file I/O or network communication. This prevents the user interface from freezing during resource-intensive operations.

- **Efficient Resource Sharing:** Threads within the same process share the same memory space and resources, such as file handles and open network connections. This efficient resource sharing can lead to reduced memory overhead and improved communication between threads.

- **Parallelism:** Threads enable parallelism, allowing multiple threads to execute tasks simultaneously. This can significantly improve the performance of applications that can benefit from dividing their work into smaller, parallel units, such as scientific simulations, multimedia processing, and data processing..

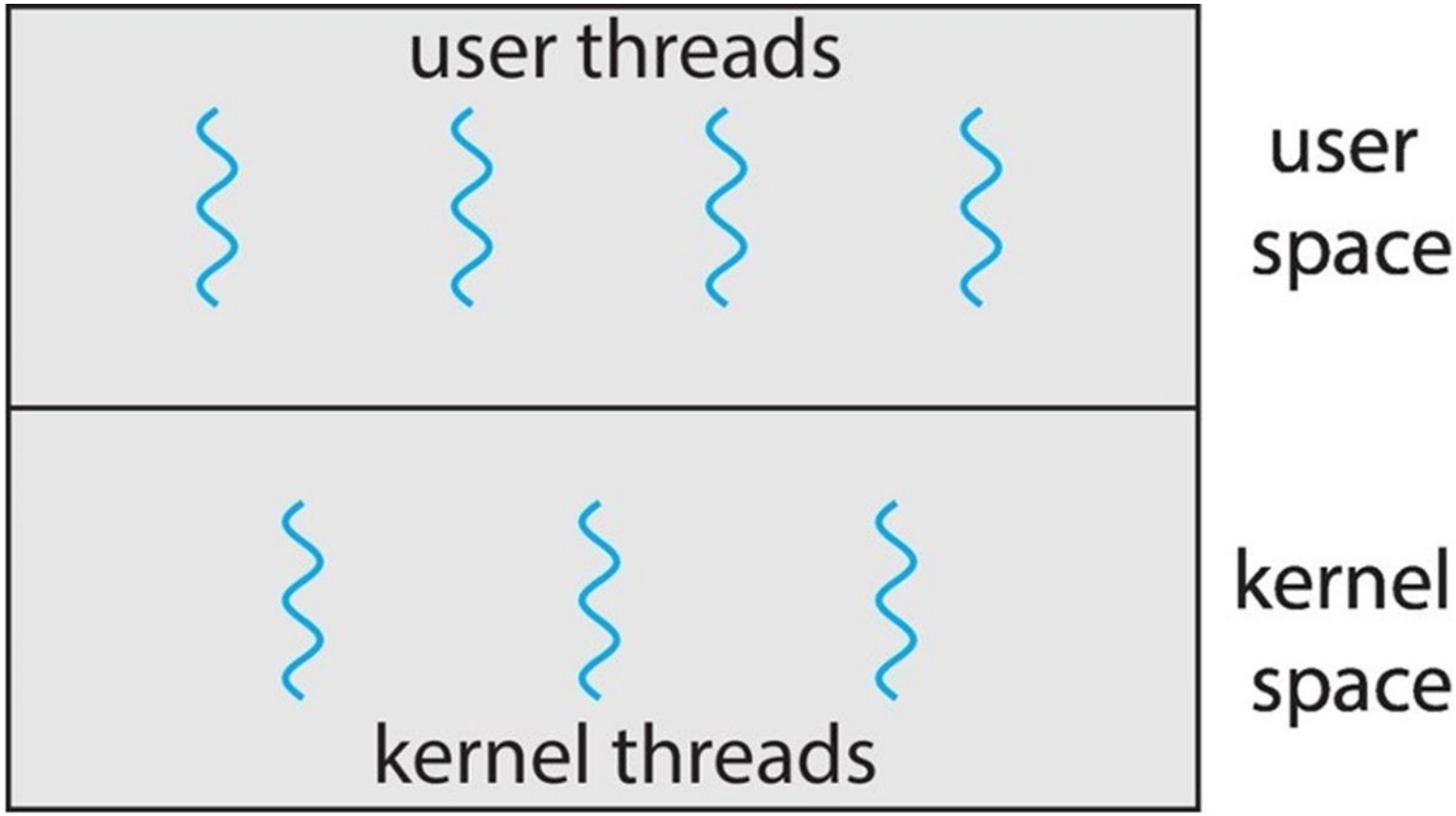
- **Enhanced Scalability:** In applications where the workload can vary, threads can be dynamically created and managed to adapt to the current demand. This scalability can improve the overall efficiency of the application.

.

User Threads and Kernel Threads

- **User threads** - management done by user-level threads library
- Three primary thread libraries:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples – virtually all general -purpose operating systems, including:
  - Windows
  - Linux
  - Mac OS X
  - iOS
  - Android

User Threads and Kernel Threads



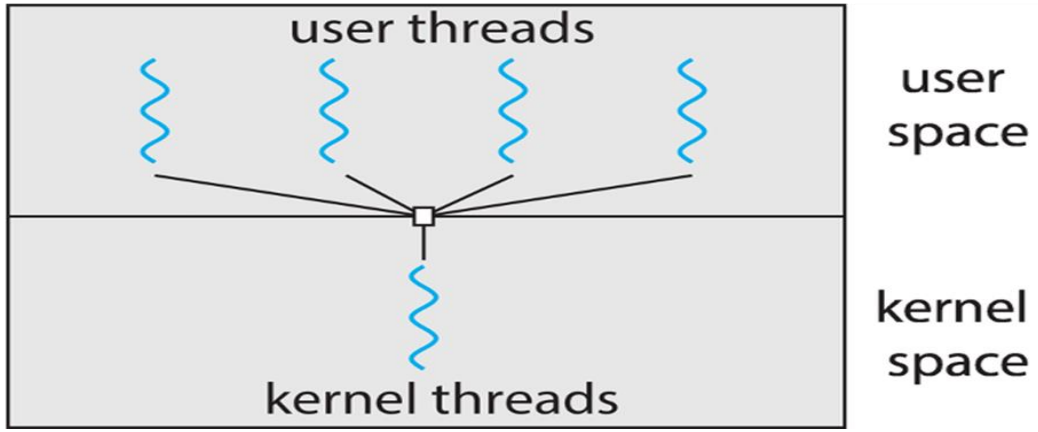
Multithreading Models

- Many-to-one
- One-to-One
- Many-to-Many
- Two-level Model

Multithreading Models

Many-to-One

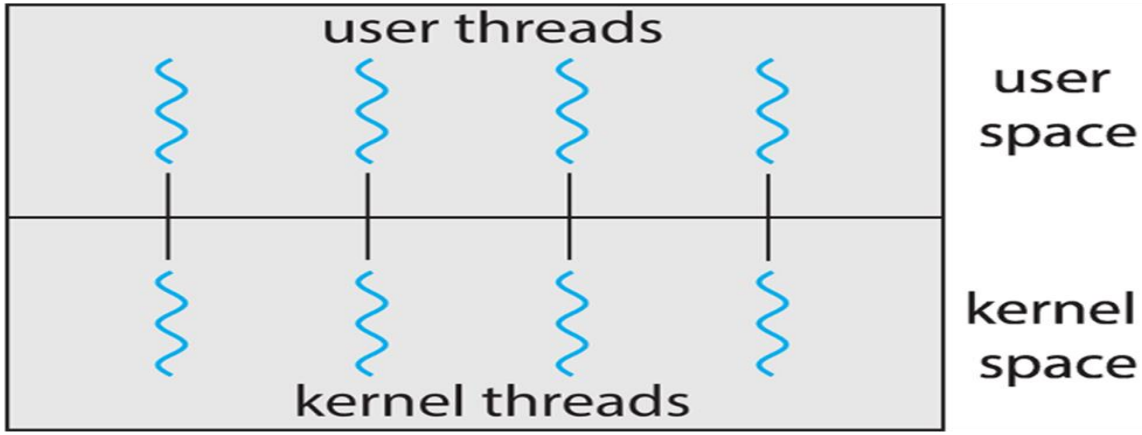
- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on muticore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**



Multithreading Models

One-to-One

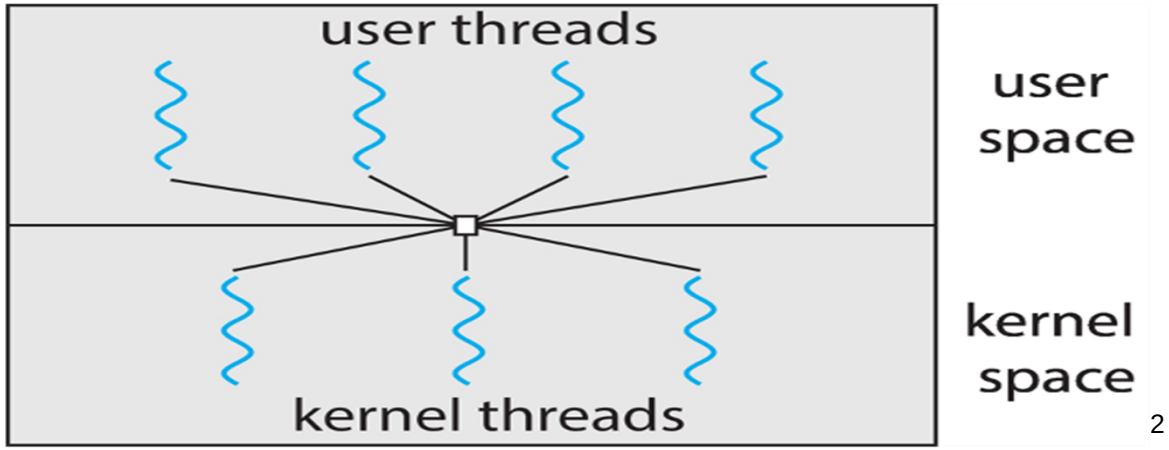
- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
  - Windows
  - Linux



Multithreading Models

Many-to-Many

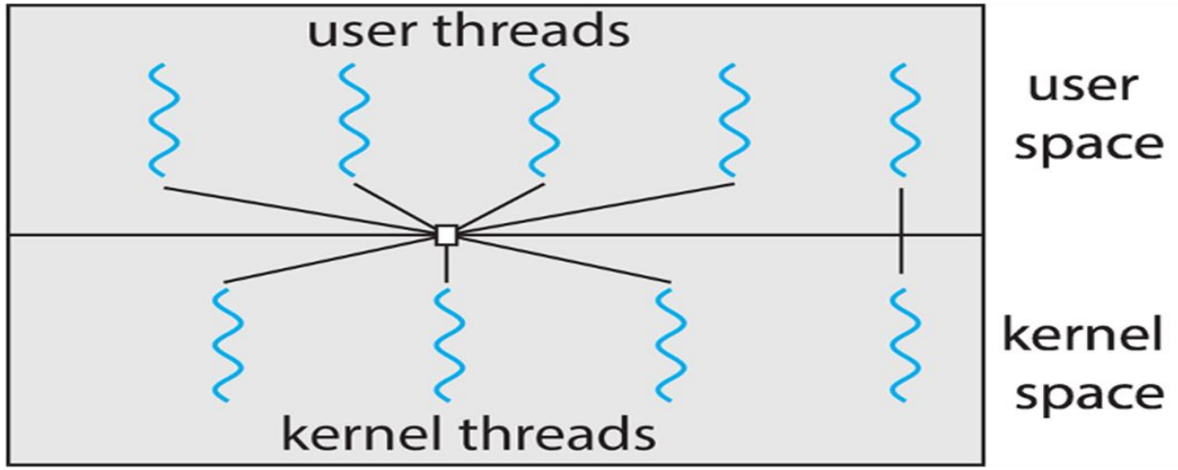
- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Windows with the *ThreadFiber* package
- Otherwise not very common



Multithreading Models

Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread

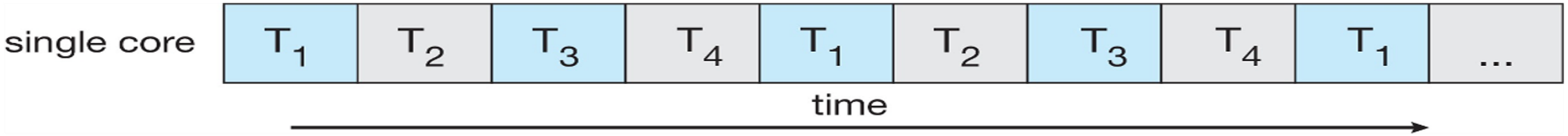


Multicore Programming

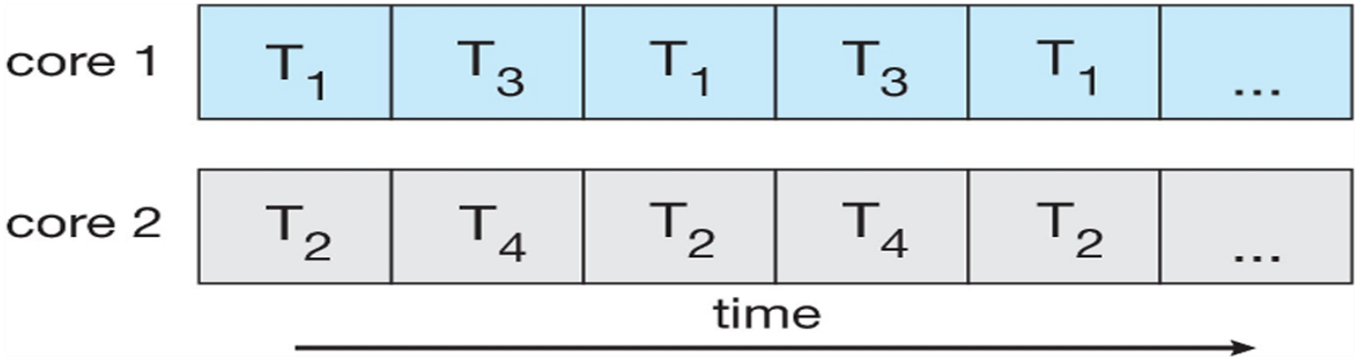
- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
  - **Dividing activities**
  - **Balance**
  - **Data splitting**
  - **Data dependency**
  - **Testing and debugging**
- ***Parallelism*** implies a system can perform more than one task simultaneously
- ***Concurrency*** supports more than one task making progress
  - Single processor / core, scheduler providing concurrency

Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**



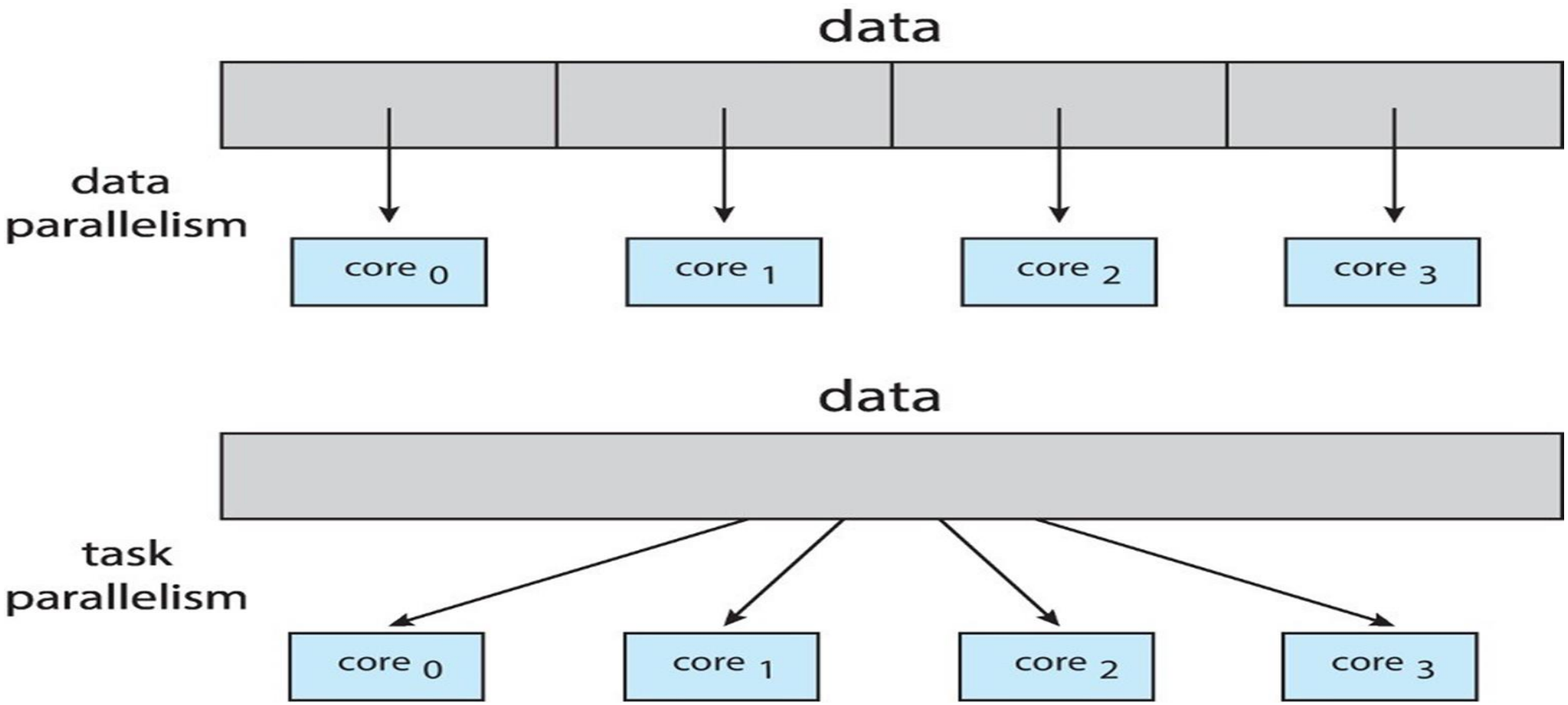
- **Parallelism on a multi-core system:**



Multicore Programming

- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
  - **Task parallelism** – distributing threads across cores, each thread performing unique operation

Data and Task Parallelism



Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- *S* is serial portion
- *N* processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As *N* approaches infinity, speedup approaches 1 / *S*

Serial portion of an application has disproportionate effect on performance gained by adding additional cores

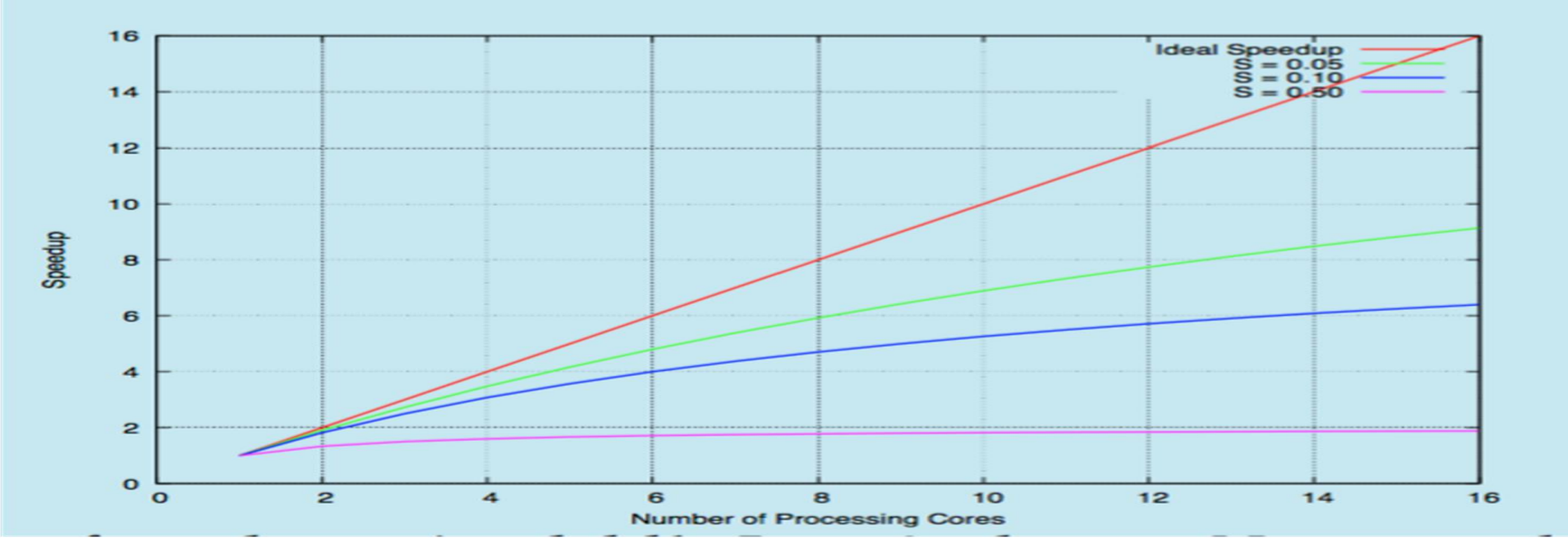
- But does the law take into account contemporary multicore systems?

Amdahl's Law

- Speedup = 1 / (S+{(1-S)/N})

= 1/ (0.25 + {(1 - 0.25)/2}

= 1/ (0.25 + 0.375) = 1/ 0.625 = 1.6



Thank You.